Introduction

This Markdown-file describes the syntax of the EBNF-language that is used to define the syntax of the languages that should be parsed.
The EBNF-syntax is the main focus of this file.
The connection to the C#-code is only explained superficially:
All chapters, except Variables and Functions > Recursions and Limitations, will only mention the C#-class that is used to construct the corresponding EBNF-expression, everything else is about the EBNF-syntax.

> Note: At the end of this file (in chapter Complete EBNF-Syntax) the complete EBNF-syntax is given from the ground up.

Helpful EBNF-Expressions

These EBNF-expressions are generally helpful for defining a syntax.
When multiple definitions are given, they start with a simple version and extend to the final definition that is used for the parser.

> Note: You might want to look at the definitions again, after you have learned how the syntax works, in order to understand the EBNF-code that is used to define the EBNF-expressions.

End of String

`e` checks if EndOfString was reached.

> Note: `e` is defined using C#-code and does not parse anything.

Natural Number

`nat` describes a whole number without a sign. (Defined using C#-code. '_' are allowed as well.)

- `nat ::= [0-9]+`
- `nat ::= [0-9] [0-9_]*`
- `nat ::= [0-9]+ ( '_' [0-9]+ )*`

White-Space

`s` is used to represent one white-space-character.

- `s ::= ' '`
- `s ::= [ \n\t]`
- define `s` using the C#-function `char.IsWhiteSpace`.

Open-Char

`openChar` is used to represent a character including escape-sequences.

- `openChar ::= ~[]`
- `openChar ::= ~[\\] | '\' ~[]`
- `openChar ::= ~[\\] | '\' ( ~[u] | 'u' [0-9a-fA-F]{4} )`

> Note: openChar is actually written with C#-code. It additionally checks and converts the escape-sequence to the corresponding character.
> Supported escape-characters: `'\0'`, `'\a'`, `'\b'`, `'\e'`, `'\f'`, `'\n'`, `'\r'`, `'\t'`, `'\v'`, `'\\'`, `'\''`, `'\"'` and `'\uXXXX'`

Comma Separated List

```
csl(arg) ::= arg ( s* ',' s* arg )*
```

Examples:

- csl of natural numbers: `csl([0-9]+)`

Atomic Expressions

Terminal (string)

```
terminal ::= '"' openChar* '"'
```

- A terminal starts and ends with double quotes.

- All characters between the double quotes describe the character-sequence which is expected by the terminal, when it's used to parse a text/code.

Examples:

- `"terminal"`
- `"Use double quotes around a text to create a terminal."`
- `"Use \\ to write special characters like \n or \t."`
- `"Use \\ to write \" inside of a terminal."`

A terminal is constructed with the C#-class `Ebnf.Expression.Terminal`.

Terminal-Char (char)

```
terminalChar ::= '\'' openChar '\''
```

- A terminal-char starts and ends with single quotes.
- Exactly one character is expected between the single quotes.
- A terminal-char is terminal that parses only one character.

Examples:

- `'a'`
- `'0'`
- `'\n'`
- `'\''`

A terminal-char is constructed with the C#-class `Ebnf.Expression.TerminalChar`.

> Note: Terminal-Char is the first extension from standard EBNF.

Character-Class

```
charClass ::= '~'? '[' openChar* ']'
```

- In order to create a negative character-class a '~' must be written in front of the character-class.
- A character-class is written with square-brackets.
- All characters inside the square-brackets will parse successfully by the character-class.

Examples:

- binary digit: `[01]`
- decimal digit: `[0-9]`
  - '-' can be used to specify that all characters in between are accepted as well.
- hexadecimal digit: `[0-9a-fA-F]`
- math-operators `[^*/+-]`
  - Remember '^' does not indicate a negative character-class.
  - When '-' is written at the start, or end of a character-class, it's counted as a normal character.
- `[0-5-9]`, `[0-5\-9]`
  - The first character-class builds 2 ranges: 0-5 and 5-9, therefore all digits will succeed.
  - The second character-class includes the digits 0 through 5 and 9 and the '-' character.
- All characters except '\n': `~[\n]`
- opening-bracket: `[([{]`
- closing-bracket: `[)\]}]`
  - In order to create a character-class that includes the closing square-bracket a '\' is needed.
- special escape-characters: `[\[\-\]]`
  - The symbols '[', '-', ']' can be written with a '\' in front, inside of a character-class.

A character-class is constructed with the C#-class `Ebnf.Expression.CharacterClass`.

Comments

```
comment ::= '#' ~[\n]*
```

- A comment starts with a '#'.
- In the current EbnfParser only one-line-comments are supported.

Combined Expressions

Since there are multiple operators in EBNF, the precedence must be specified. Thus the second definition is given.

> Note: In this chapter Combined Expressions some definitions are written in brackets. These definitions are only given for explanation-purposes.

Choices

```
( choice ::= expr ( s* '|' s* expr )* )
expr1 ::= expr2 ( s* '|' s* expr2 )*
```

- '|' can be used to allow multiple EBNF-expressions.
- A choice will go through each choice from left to right and stops when a successful choice was found.

> Note: The order matters.
> `"sub" | "substring"` The second choice will never be successful, because the first choice will be checked first.
> possible solutions: `"substring" | "sub"` or `"sub" "string"?`
> Note: the second solution is preferred, since in the first solution would parse the string "sub" twice.

A choice is constructed with the C#-class `Ebnf.Expression.Choice`.

Sequence

```
( sequence ::= expr ( s+ expr notDef )* )
expr2 ::= expr3 ( s+ expr3 notDef )*
```

- In order to concatenate multiple EBNF-expressions they can simply be written in the desired order.
- `notDef` is a special expression that ensures that the parsed expression is not part of the next definition.
  - `firstDef ::= a b | c secondDef ::= a (b | c)`
  - In the example above, `notDef` would stop the parsing of the first definition at `secondDef`, since it is followed by `::=`.
  - Note: `notDef` is defined using C#-code. It works like a negative lookahead: `s* "::="`

> Note: At least one white-space-character is needed to separate the individual expressions.

A sequence is constructed with the C#-class `Ebnf.Expression.Sequence`.

Repetition

```
repetition ::= [*+?] | '{' nat ( ',' nat? )? '}'
```

- In order to repeat EBNF-expression, repetition-postfixes can be used.

Explanation of the possible postfixes:

- `{min,max}` : This is the general case. `min` specifies the minimum number of how often the expression must succeed. `max` specifies the limit, at which the EBNF-expression will stop.
- `{num}` = `{num,num}` : The expression must succeed exactly `num` times.
- `{min,}` = `{min,int.MaxValue}` : The expression must succeed at least `min` times. Since every repetition-expression is expressed in the general case, which uses 32-bit signed integers, the maximum is set to the largest number that can be represented with this data-type. (`int.MaxValue = 2^31 - 1 = 2 147 483 647`)
- `*` = `{0,}` : zero or more
- `+` = `{1,}` : one or more
- `?` = `{0,1}` : optional (zero or one)

A repetition is constructed with the C#-class `Ebnf.Expression.Repetition` .

> Note: To construct the repetition, the EBNF-expression must be given as well.

Extension

```
extension ::= [/!] name
expr3 ::= expr4 ( repetition | extension )*
```

- In order to simplify processing the parsed result extensions are implemented in the EBNF-language.
- Extensions allow you to process and check sub-expressions.
  e.g.: `[0-9]+/checkNumLessThan1024`
- Extensions also are written as a postfix. Thus second definition of repetition is combined with extensions.
- Extensions start with `'/'` or `'!'` to separate from the expression and to specify if a success or an error should be handled.
- `'/'` : The expression will be handled by the extension, if it was successful.
- `'!'` : The expression will be handled by the extension, if an error occurred.

An extension can be constructed in multiple ways:

- Specify the C#-code for the extension with the abstract class `Ebnf.PostProcess` . Use `Ebnf.SuccessProcess` or `Ebnf.ErrorProcess` .

- Use `Ebnf.Parser().AddSuccessProcess(...)` or `Ebnf.Parser().AddErrorProcess(...)` when writing EBNF-expressions using EBNF-code. (Extensions from EBNF-code are constructed with the C#-class `Ebnf.Expressions.PostProcessingReference`)
- Use `Ebnf.Expression().Extend(...)` when writing EBNF-expression using C#-code. (Extensions from C#-code are constructed with the C#-class `Ebnf.Expression.PostProcessingExpression` .)

> Note: Additional information about processing subexpressions can be found in the chapter Variables.

Arguments

```
arguments ::= '(' ( s* csl(expr) s* )? ')'
expr4 ::= expr5 arguments?
```

You may have noticed that the comma-separated-list EBNF-expression is special.
It allows to specify parts of its definition by providing EBNF-expressions as arguments.
This feature was actually implemented and can be used in the EBNF-code.
You will learn more about it in the chapter Variables and Functions > Variable.

> Note: that's the reason why at least one whitespace is necessary when creating a EBNF-Sequence.

Function-calls are constructed with the C#-class `Ebnf.Expression.Application` .

Brackets

```
bracket ::= '(' expr ')'
atomic ::= terminal | terminalChar | charClass | variable
expr5 ::= bracket | atomic
```

- You may have noticed, that even though all numbered expressions (`expr1` ... `expr5`) are defined, in some non-exemplary definitions the yet undefined variable `expr` is used.
  (For now `expr` = `expr1` holds.)
  `expr` will be defined in the later chapter Variables and Functions > Lambda

Variables and Functions

Variable

```
name ::= [a-zA-Z] [a-zA-Z0-9_]*
variable ::= name
```

You may have noticed that a variable can be defined using this EBNF-expression:

```
name s* "::=" s* expr
```

And you probably also know how to refer to a variable inside of an EBNF-expression. (You just write the name of the variable.)
And you may have noticed that extensions are references as well.
A variable does not only store an expression, it also stores a successProcess and an errorProcess.
The following points will point out what can be done with variables:

- When defining a variable and assigning a postProcess to the same name. This process will automatically be applied to the result of the expression.
- The variable-name can still be used as an extension.
- In case you want to apply an extension to the whole expression of a variable this syntax can be used, instead of using brackets:

  ```
  name extension* s* "::=" s* expr
  ```

  This is a syntax-sugar. It will compile to the same expression as if you would have used this syntax:

  ```
  name s* "::=" s* '(' expr ')' extension*
  ```

  (The order will stay the same: `name/extA/extB ::= a b c` == `name ::= (a b c)/extA/extB`)

Additionally, when multiple definitions are given for the same variable, the definitions will be combined to a choice-expression.

A variable is constructed with the C#-class `Ebnf.Variable` .

> Note: "Variable" might not be the best name for most situations, since expressions are normally only assigned to variables once and then never changed again.

Lambda

```
lambda ::= ( name | parameters ) s* "-->" s* expr
```

> Note: Since expr1 does not include lambda-expressions, expr must be defined like this:
> `expr ::= expr1 | lambda` .

EBNF-Functions like csl (comma-separated-list) can be defined using lambda-expressions.
A lambda-expression starts with the parameter-list followed by an arrow and ends with the EBNF-expression that should be used when the function is called.

- In case of a lambda-expression with only one parameter, the parameter can be written without brackets.
  (Using `name` instead of `parameters` as the parsing-rule for the parameter-list.)
- Otherwise multiple parameters can be written as a comma-separated-list in brackets.
  (Using `parameters` as the parsing-rule for the parameter-list.)
  (For now `parameters` can be defined as `'(' s* csl(name) s* ')'`)
- Additionally the parameter-list can be empty. In this case empty brackets must be written for the parameter-list.
  (Extending the definition of `parameters` to `'(' ( s* csl(name) s* )? ')'`)

Examples:

- comma-separated-list: `csl ::= arg --> arg ( s* ',' s* arg )*`
- separated-list: `sl ::= (arg, sep) --> arg ( s* sep s* arg )*`
- list-of-numbers: `myList ::= () --> [0-9]+ ( s* ',' s* [0-9]+ )*`

> Note: A lambda-expression with an empty parameter-list is basically a variable that stores an EBNF-expression.
> The difference is that additional () must be written when referring to the expression. (`myList` vs `myList()`)
> This distinction can be used when defining a variable like this: `myExpr ::= exprA | () --> exprB`
> Note: `myExpr` refers to the expression `exprA` and `myExpr()` refers to the expression `exprB` .
> The same principle can be used for overloading variables:
> `myExpr ::= () --> exprA | x --> exprB(x) | (x, y) --> exprC(x, y)`

> Note: When creating a lambda-expression, the parameter-names must be unique. (Otherwise an exception will be thrown when parsing the EBNF-code.)
> Note: When nesting lambda-expressions, like this:
> `myExpr ::= (x, y) --> ((x, z) --> expr(x, y, z))(a, b)`
> The inner lambda-expression will cover the parameter x of the outer lambda-expression. Thus the parameter x of the outer lambda-expression will not be accessible from the body of the inner lambda-expression.

Optional Parameters

```
parameters ::= '(' (
    s* csl(name) s*
    ( '=' s* expr s* ( ',' s* name s* '=' s* expr s* )* )?
)? ')'
```

- You might have come up with the idea to use overloading for default parameters. But this is not necessary, since optional parameters can be defined in the EBNF-language as well.
- Optional parameters can be defined by using an equal sign '='.

> Note: Like in C# optional parameters must be written at the end of the parameter-list.
> Note: Normally it wouldn't make sense to use () when calling a function, since the expression could be defined directly.
> But when using only optional parameters, using () might occur more often.

Examples:

- separated-list using overloading:
  `sl ::= csl | (arg, sep) --> arg ( s* sep s* arg )*`
- separated-list with default separator:
  `sl ::= (arg, sep = ',') --> arg ( s* sep s* arg )*`
- separated-list with default separator and default argument:
  `sl ::= (arg = [0-9]+, sep = ',') --> arg ( s* sep s* arg )*`

Optional parameters are realized by using the C#-class `Ebnf.Expression.OptionalParams`.

> Note: Overloading is realized by using choices. Thus it might happen that some functions will be covered by earlier choices.

Local Variables

Lambda-expressions can also be called directly inside of EBNF-expressions. Thus they can be used to define local variables. But there is also a special syntax for defining local variables.

```
lambda ::= csl(name s* '=' s* expr) s* ';' s* expr
```

Examples:

- List of numbers with lambda-expression as local variable:
  - `myList ::= (num --> num ( s* ',' s* num )*)([0-9]+)`

    - `myList ::= ((num, sep) --> num ( s* sep s* num )*)([0-9]+, ',')`
- List of numbers with local variable:
    - `myList ::= num = [0-9]+; num ( s* ',' s* num )*`
    - `myList ::= num = [0-9]+, sep = ','; num ( s* sep s* num )*`

> Note: Do you see the difference between the following two definitions?
> `myList ::= num = [0-9]+, sep = ','; num ( s* sep s* num )*`
> `myList ::= num = [0-9]+; sep = ','; num ( s* sep s* num )*`
> Both definitions are logically the same, but the second definition will create an unnecessary capsulation, since it unfolds to a nested lambda-expression. Thus the first definition is preferred. Here are both definitions in their unfolded form:
> `myList ::= ((num, sep) --> num ( s* sep s* num )*)([0-9]+, ',')`
> `myList ::= (num --> (sep --> num ( s* sep s* num )*)(','))([0-9]+)`

> Note: Local variables compile to the same expression as lambda-expressions with arguments. Therefore no additional C#-class is needed.

Functions

`definition ::= name parameters? extension* s* "::=" s* expr`
Like with extensions, there is a syntax-sugar for defining functions.

Examples:

- comma-separated-list with lambda:
  `csl ::= arg --> arg ( s* ',' s* arg )*`
- comma-separated-list as function:
  `csl(arg) ::= arg ( s* ',' s* arg )*`

> Note: Both definitions compile to the same expression.

> Note: This syntax-sugar is only possible for global variables, not for local variables.

Recursions and Limitations

Clearly, recursion is needed to define a syntax, where infinite nesting is possible. When creating recursion using variables, there is no problem, since the variable can refer to itself. But there arise some problems when using local variables.

In order to get a better understanding of the possibilities and limitations of this EBNF-language, I will list out some examples:

- X `myExpr ::= x = '(' x ')'; x`
  Here is an example of a limitation. (local variables cannot refer to themselves.)
  Remember, local variables are only a syntax-sugar for lambda-expressions with arguments. Thus the example above would unfold to this:
  `myExpr ::= (x --> x)('(' x ')')`
  Now, it should be clear that the x in the argument either is undefined, or refers to a global variable.
  When recursion is needed use global variables instead:
  `myExpr ::= '(' myExpr ')'`
- ✓ `app(f, x) ::= f(x)`
  Here is an example of a possibility.
  Since a lambda-expression is an expression like any other, it is possible to pass functions as arguments to other functions.
- X `add(x) ::= y --> x y` or `add ::= x --> y --> x y`
  Here is an example of a limitation.
  `add(a)(b)` would be expected to parse as `a b`.
  But when applying an argument to a function, the result is a closed expression. Therefore no further arguments can be passed to the result.
  But to achieve the same result, the example can be modified like this:
  `add(x, y) ::= x y`
  `mySubFunc(y) ::= add(a, y)`
- X `myFunc(a, b) ::= a b e | myFunc(a b, b a)`
  Here is another example of a limitation. (Applying the parameters of a function to itself results in an infinite loop.)
  When a function is created, the parameters are connected to the expression. When the function is called, for example with `'0'` and `'1'`, the parameters will be loaded with these arguments. Now the function will try to parse using `'0' '1' e` first. If successful no problem occurs. But when the second choice is checked, this happens:
  The parameters `(a, b)` will be loaded with `(a b, b a)`.
  Since `a` and `b` are references to the arguments, the reference to the original arguments `'0'` and `'1'` is lost.
  Now `(a b) (b a)` will be used to parse.
  Since `a` refers to `a b` an infinite loop occurs:
  `a b = (a b) b = ((a b) b) = (((a b) b) b) = ...`

Complete EBNF-Syntax

- `e` checks if EndOfString was reached. (Defined using C#-code)
- `s` is a white-space-character. (Defined using C#-code `char.IsWhiteSpace`)
- `openChar` is a character including escape-sequences. (Defined using C#-code)
  The code can be represented like this EBNF-expression:
  `~[\\] | '\\' ( [0abefnrtv\\'"] | 'u' [0-9a-fA-F]{4} )`
- `nat` describes a whole number without a sign. (Defined using C#-code. '_' are allowed as well.)
  The code can be represented like this EBNF-expression:
  `[0-9]+ ( '_' [0-9]+ )*`
- `notDef` is a special expression that ensures that the parsed expression is not part of the next definition. (Defined using C#-code. It works like a negative lookahead: `s* "::="`)
- `asTerminal` is a SuccessProcess that excludes `"\""` .
- `asTerminalChar` is a SuccessProcess that excludes `"'"` .
- `asCharClass` is a SuccessProcess that excludes `"]"` and an ErrorProcess that includes `"\["` , `"\-"` and `"\]"` .

```
name ::= [a-zA-Z] [a-zA-Z0-9_]*
csl(arg) ::= arg ( s* ',' s* arg )*

comment ::= '#' ~[\n]*
definition ::= name parameters? extension* s* "::=" s* expr
main ::= s* ( (definition | comment) s* )* e

lambda ::= csl(name s* '=' s* expr) s* ';' s* expr
lambda ::= ( name | parameters ) s* "-->" s* expr
parameters ::= '(' (
    s* csl(name) s*
    ( '=' s* expr s* ( ',' s* name s* '=' s* expr s* )* )?
)? ')'
arguments ::= '(' ( s* csl(expr) s* )? ')'

repetition ::= [*+?] | '{' nat ( ',' nat? )? '}'
extension ::= [/!] name

expr ::= expr1 | lambda
expr1 ::= expr2 ( s* '|' s* expr2 )*
expr2 ::= expr3 ( s+ expr3 notDef )*
expr3 ::= expr4 ( repetition | extension )*
expr4 ::= expr5 arguments?
expr5 ::= '(' expr ')' | atomic

atomic ::= terminal | terminalChar | charClass | variable
terminal ::= '"' openChar/asTerminal* '"'
terminalChar ::= '\'' openChar/asTerminalChar* '\''
charClass ::= '~'? '[' openChar/asCharClass!asCharClass* ']'
variable ::= name
```

> Note: `main` is the starting rule for the EBNF-syntax.